

Plan de Pruebas IMSGA (React Native / Expo)

1. Introducción

Objetivo: documentar la estrategia, alcance y cobertura real de la suite de pruebas automatizadas del proyecto IMSGA, una app React Native/Expo para la gestión operativa de despachos de ambulancia, inventario de insumos, atenciones clínicas y personal de un servicio de emergencias.

Alcance: la suite de pruebas cubre tres capas:

- 1.- Lógica de contexto (estado global vía React Context + fetch a la API).
- 2.- Guards de routing/rol (expo-router)
- 3.- Componentes de formulario con lógica no trivial (validación, cálculos, filtrado).

No cubre pantallas completas ni componentes puramente presentacionales (ver sección 5).

Stack de testeo: Jest con preset jest-expo + @testing-library/react-native. Configuración en jest.config.js (raíz del repo).

Los tests viven en __tests__/, organizados en utils/, context/, app/ y components/.

Patrones de mocking establecidos:

- Contexts que hacen fetch: mock de @/context/AuthContext (fetchConSesion, useAuth) vía jest.mock.
- Layouts con guards de rol: mock de expo-router (Redirect/Tabs) y @expo/vector-icons.
- Componentes con formularios: un "Harness" que envuelve el componente en useForm()/FormProvider real de react-hook-form.

2. Estrategia de priorización

La suite se construyó en dos pasadas priorizando por riesgo de negocio, no por porcentaje de cobertura total. El criterio de cada capa es el siguiente:

Capa 1 : Alto riesgo, prioridad de cobertura

Lógica donde una regresión silenciosa permite un bypass de autenticación, corrompe un registro clínico/operativo, o afecta stock.

Incluye: guards de rol y sesión (quién puede ver/hacer qué), contexts que manejan despachos, inventario, atenciones y pacientes, y los componentes que escriben directamente sobre esos datos (signos vitales, datos de paciente).

Capa 2 : Lógica real de menor riesgo

Formularios y flujos con ramas reales (validación, normalización, estados de error) pero cuyo radio de impacto si falla es menor: no hay bypass de autenticación ni pérdida de datos clínicos/operativos, a lo sumo una mala experiencia de usuario recuperable (un login que reintenta, una notificación que no se descarta).

Capa 3 : Deliberadamente excluido

Código sin ramas que puedan romperse (mapas de constantes, componentes presentacionales sin condicionales), o side-effects de módulos nativos donde un mock pesado da menos garantía real que una verificación manual/E2E (PDF, Firebase). Se documenta explícitamente en la sección 5 para distinguir "decisión consciente" de "hueco olvidado".

3. Matriz de pruebas

3.1 Utils

Archivo de test	Qué unidad cubre	Casos principales	Capa
utils/format.test.ts	formatearRut, validarRut, validarFecha, formatearFecha, formatearTelefono, validarTelefono, formatearHora, formatearFechaHora	Éxito: formato limpio con puntos/guion, verificador k, DD-MM-YYYY, +569, HH:MM, fecha ISO a locale CL. Edge: valor crudo si es muy corto, dígito verificador inválido, mes/día inválido, año futuro, input vacío/undefined.	Capa 2
utils/labels.test.ts	traducirRol	Éxito: traduce medic/nurse/control/driver. Edge: rol desconocido o vacío cae a "Usuario".	Capa 2

(sin archivo dedicado — cubierto vía AuthContext.test.tsx)	utils/api.ts: fetchConSesion, clearSession, setSessionExpiredHandler (re-exportados por context/AuthContext)	Éxito: headers de sesión/CSRF, persistencia de set-cookie. Error: 401 limpia storage y dispara handler de sesión expirada.	Capa 1
--	--	---	--------

3.2 Contexts

Archivo de test	Qué unidad cubre	Casos principales	Capa
AuthContext.test.tsx	fetchConSesion, useAuth (login, totpValid, logout, restauración de sesión)	Éxito: login válido, totp con rol+personalId, restaura sesión desde AsyncStorage. Error: credenciales rechazadas, error de red, sin credenciales pendientes. Edge: no hay sesión guardada al montar.	Capa 1
DespachosContext.test.tsx	fetchDespachos, actualizarDespacho, seleccionarDespacho, limpiarDespachoActivo, despachosPorPersonal, setDespachoActivoPorUsuario	Éxito: carga inicial, refetch ante notificación push nueva, actualiza solo el despacho objetivo. Error: fetch falla → error + arreglo vacío. Edge: no duplica refetch para la misma notificación, id desconocido → null, personalId sin despacho activo → null/vacío.	Capa 1
InventoryContext.test.tsx	fetchInsumos, agregarInsumos, buscarInsumo, actualizarStock, moverInsumo	Éxito: carga y mapea insumos, POST/PATCH seguido de refetch. Error: 409 por stock insuficiente lanza mensaje específico, otras fallas lanzan mensaje genérico. Edge: búsqueda case-insensitive sin match → vacío.	Capa 1
AtencionContext.test.tsx	agregarAtencion, fetchAtenciones, fetchAtencionDetalle	Éxito: POST de payload transformado, mapeo de resumenAtenciones, descarga de documento S3 con sello fusionado. Error: 409 por despacho cerrado/sin stock lanza mensaje específico, otras fallas mensaje genérico, falla de descarga S3 retorna null + error.	Capa 1
AmbulanciaContext.test.tsx	Carga de ambulancias	Éxito: popula desde la API al montar. Error: fetch falla y error de red → error + arreglo vacío.	Capa 1
PacienteContext.test.tsx	Carga de pacientes, agregarPaciente, buscarPaciente	Éxito: carga al montar si hay usuario, agrega paciente a estado local, encuentra por rut. Edge: no hace fetch sin usuario, búsqueda sin	Capa 1

		match → undefined. Error: fetch falla y se registra log.	
GrupoContext.test.tsx	fetchGrupos, crearGrupo, agregarMiembro, removerMiembro	Éxito: carga grupos, POST de grupo nuevo + refetch, suscribir/desuscribir miembro. Error: fetch falla y queda log, falla de creación lanza el mensaje de la API, falla de suscripción/desuscripción lanza error con status.	Capa 2
PersonalContext.test.tsx	Carga de personal, actualizarDisponibilidad, registrarWorker	Éxito: carga al montar, marca solo el personal objetivo como inactivo, registra worker y retorna credenciales creadas. Error: fetch falla y queda log, registro falla y lanza el mensaje de la API.	Capa 2
NotificationContext.test.tsx	Cola de notificaciones FCM, dismissNotification, markAllRead	Éxito: agrega mensaje entrante al frente de la lista y suma no-leídas, marca todas como leídas. Edge: genera id propio si falta messageId, descarta solo la notificación objetivo.	Capa 2

3.3 Layouts / routing (guards de rol)

Archivo de test	Qué unidad cubre	Casos principales	Capa
app/index.test.tsx	app/index.tsx — routing inicial	Éxito: redirige rol control a AdminDashboard, roles no-control a UserDashboard. Edge: muestra loader mientras se restaura la sesión, redirige a login sin usuario autenticado.	Capa 1
app/admin-layout.test.tsx	app/(admin)/_layout.tsx — guard de rol admin	Éxito: renderiza tabs de admin para rol control. Error/edge: redirige a login sin usuario o si el usuario no es control.	Capa 1

app/user-layout.test.tsx	app/(user)/_layout.tsx — guard de rol user	Éxito: oculta el tab registrar-atencion para rol driver. Error/edge: redirige a login sin usuario o si el usuario es control.	Capa 1
---------------------------------	---	--	--------

3.4 Componentes

Archivo de test	Qué unidad cubre	Casos principales	Capa
LoginForm.test.tsx	Formulario de login	Éxito: limpia puntos del rut, hace login, guarda credenciales pendientes y navega a totp. Error: campos vacíos no llaman login, credenciales rechazadas no navegan. Edge: navega a recuperación al presionar "Olvidé mi contraseña".	Capa 2
InsumosForm.test.tsx	Búsqueda/alta de insumos + DosisInput	Éxito: filtra por nombre acotado a la ambulancia del despacho activo, agrega insumo con dosis 1 por defecto. Error/edge: evita duplicados, sin resultados muestra mensaje, DosisInput limpia caracteres no numéricos al tipear y resetea a 1 en blur si queda vacío o bajo 1.	Capa 2
ControlVitales.test.tsx	Registro de controles de signos vitales	Éxito: agrega control con hora actual y vitales en cero, pagina entre múltiples controles. Cálculo: $PAM = \text{round}((PAD * 2 + PAS) / 3)$. Edge: estado vacío sin controles.	Capa 1
FormPaciente.test.tsx	Formulario de paciente (variante user)	Éxito: precarga y deshabilita campos si el despacho ya tiene paciente asociado, formatea el rut mientras se tipea sin paciente precargado. Edge: deja campos editables/vacíos sin paciente, direccionDestino siempre deshabilitada y sincronizada desde el despacho activo.	Capa 1

4. Cobertura actual

Resultado real de `npx jest --coverage --silent` sobre la raíz del repo (18 suites, 139 tests, todos en verde):

Archivo	% Stmts	% Branch	% Funcs	% Líneas	Líneas sin cubrir
Todos	84.06	69.36	79.32	85.88	

app / index.tsx	100	100	100	100	
app/(admin) / _layout.tsx	60	100	20	55.55	37-75
app/(user) / _layout.tsx	70	100	25	70	43-62
components / LoginForm.tsx	100	83.33	100	100	69-73
components/user / ControlVitales.tsx	80.95	66.66	71.42	80	96,113,130,146,163,179,196,216
components/user / FormPaciente.tsx	100	75.6	100	100	26,37,45-48,61,79,101,185,206
components/user / InsumosForm.tsx	97.5	68.75	93.75	97.14	153
context / AmbulanciaContext.tsx	86.48	50	85.71	87.87	59-68
context / AtencionContext.tsx	92.85	76.66	81.25	97.33	35-36
context / AuthContext.tsx	80.86	50.94	62.5	85.43	81,91-113,193-194,220,245-246,255
context / DespachosContext.tsx	51.42	41.86	62.06	51.26	28,113-115,126-128,134-135,151-165,171-258
context / GrupoContext.tsx	93.02	88.88	70	100	28,41
context / InventoryContext.tsx	84.5	70	87.5	85.24	52-53,58-78
context / NotificationContext.tsx	100	100	100	100	
context / PacienteContext.tsx	100	87.5	100	100	41
context / PersonalContext.tsx	96.77	90	90	100	32
data/constants, data/mock, styles	100	100	100	100	
utils / api.ts	100	92.3	100	100	25

utils / format.ts	96.96	93.33	100	100	24,44,73
utils / labels.ts	100	100	100	100	

Interpretación:

- Fuerte: utils (96-100% en las tres métricas), components/* (97-100% stmts), NotificationContext, PacienteContext y app/index.tsx en 100%. La lógica que más se manipula a mano (formularios, cálculos, parsing) es la más cubierta.
- Débil y esperado: app/(admin) y app/(user) _layout.tsx tienen funciones al 20-25% porque los tests cubren el guard de rol (el código de alto riesgo) pero no el cuerpo completo de renderizado de tabs (líneas 37-75 / 43-62) — coherente con el criterio de Tier 1 centrado en el guard, no en el layout visual completo.
- Débil y a revisar: DespachosContext.tsx es el archivo más bajo de la suite (51.42% stmts, 41.86% branch), con un bloque grande sin cubrir (líneas 151-258). Indica que existen métodos o ramas del context (probablemente mutaciones o selectores adicionales) fuera del alcance de la pasada actual.
- AuthContext (80.86% stmts, 50.94% branch) y AmbulanciaContext (86.48% stmts, 50% branch) tienen branch coverage notablemente más bajo que statement coverage, señal de ramas de error/edge sin ejercitar (p. ej. variantes de respuesta de error no cubiertas).

5. Exclusiones deliberadas (Capa 3)

Las siguientes exclusiones son decisiones explícitas.

5.1 Mapas de constantes sin lógica

- `utils/despacho.ts`: mapa de constantes, sin ramas ni lógica que pueda romperse.

5.2 Side-effects de módulos nativos

- `utils/pdf.ts`, `utils/firebaseApp.ts`, `utils/firebaseMessaging.ts`: requieren mockear pesadamente APIs nativas (generación de PDF, inicialización de Firebase/FCM). Un mock de esa magnitud da menos garantía real que una verificación manual o E2E sobre el dispositivo/build real.

5.3 Componentes puramente presentacionales

- `EstadoBadge`, `AppHeader`, `DashboardHeader`, `NotificationCard`, `DispatchCard`, `RecoverConfirm`: sin ramas condicionales relevantes renderizan props sin lógica que pueda fallar de forma silenciosa.

5.4 Tests de pantalla completa

- Las 27 rutas bajo `app/`: la lógica de negocio ya está cubierta a nivel de context y de componente. Duplicarla en tests de pantalla completa es alto costo (mocks extensos de navegación + providers) y bajo valor incremental, dado que no agrega ninguna rama nueva no cubierta.

5.5 Variantes "admin" con mismo molde que "user"

- Variantes admin de `FormPaciente/FormDespacho` e `InsumosForm` avanzado: comparten el mismo molde de lógica ya validado en la variante user.